ACTIVIDAD PARCIAL 2

# Modelado de problemas de búsqueda

Representación de estados, acciones, costos y metas mediante grafos

| | |
|---|---|
| **Universidad** | Politécnica de Atlautla |
| **Programa educativo** | Ingeniería en Tecnologías de la Información e Innovación Digital |
| **Asignatura** | Fundamentos de Inteligencia Artificial |
| **Estudiante** | Luis Antonio Roman Rey |
| **Docente** | Mtro. Javier García Lara |
| **Grupo / cuatrimestre** | 171 y 172 / 7.º cuatrimestre |
| **Periodo** | 12 al 26 de octubre de 2026 |
| **Fecha de entrega indicada** | 18 de octubre de 2026 |
| **Sección** | 2.1 Problemas de búsqueda |

Documento individual elaborado con estructura académica, análisis técnico, conclusiones y referencias en formato APA 7.

# 1. Introducción

En inteligencia artificial, un problema de búsqueda se define mediante un estado inicial, un conjunto de acciones, un modelo de transición, una prueba de objetivo y, cuando corresponde, un costo de camino. El algoritmo explora alternativas para encontrar una secuencia de acciones que alcance el objetivo. Este trabajo modela dos situaciones realistas: la entrega de un paquete en una red de ubicaciones y la salida de un laberinto.

# 2. Objetivo

Formalizar dos problemas como grafos, implementar algoritmos de búsqueda en Python, comparar las soluciones y discutir la complejidad temporal y espacial de los métodos utilizados.

# 3. Conceptos y formulación

**Estado:** descripción de la situación actual. **Acción:** movimiento permitido desde un estado. **Modelo de transición:** estado al que conduce una acción. **Prueba de objetivo:** condición que determina si se alcanzó la meta. **Costo:** valor acumulado de una ruta, como distancia o tiempo. **Solución:** secuencia de acciones que lleva del estado inicial a la meta.

# 4. Problema A — Ruta de reparto entre ubicaciones

**Escenario:** un repartidor debe trasladar un paquete desde el almacén hasta el destino D a través de una red simplificada de ubicaciones. Cada conexión tiene una distancia estimada en kilómetros. El objetivo es minimizar la distancia total, no simplemente el número de tramos.

**Estado:** ubicación actual del repartidor (A, B, C, D o E).
**Estado inicial:** A (almacén).
**Acciones:** desplazarse a un nodo conectado por una arista.
**Transición:** cambiar de la ubicación actual al vecino seleccionado.
**Costo de acción:** kilómetros estimados de la conexión.
**Prueba de objetivo:** ubicación actual = D.
**Función de costo:** suma de kilómetros de todos los tramos recorridos.

**Grafo ponderado de ejemplo:** A–B = 4 km; A–C = 2 km; C–B = 1 km; B–D = 5 km; C–D = 8 km; C–E = 10 km; E–D = 2 km. Las conexiones se modelan como bidireccionales para este escenario didáctico. Los pesos son ficticios y se usan únicamente para demostrar el algoritmo.

Solución esperada: A → C → B → D, con costo 2 + 1 + 5 = 8 km. La alternativa A → B → D cuesta 9 km; A → C → D cuesta 10 km. Dijkstra garantiza la ruta de costo mínimo cuando todos los pesos son no negativos.

# 5. Problema B — Salida de un laberinto

**Escenario:** un agente se mueve por una cuadrícula donde algunas celdas son paredes. Debe ir desde S (inicio) hasta G (meta). Cada movimiento válido cuesta una unidad.

**Estado:** coordenada (fila, columna) de la celda actual.
**Estado inicial:** S = (0, 0).
**Acciones:** mover arriba, abajo, izquierda o derecha, siempre que la celda esté dentro de la cuadrícula y no sea una pared.
**Transición:** actualizar la coordenada a la celda vecina permitida.
**Costo de acción:** 1 por movimiento.
**Prueba de objetivo:** coordenada actual = G.
**Función de costo:** cantidad de movimientos realizados.

La implementación usa una cuadrícula de cinco filas por seis columnas. Las paredes bloquean el paso; las demás celdas son transitables. BFS (búsqueda en anchura) encuentra una ruta con el menor número de movimientos cuando cada paso

tiene el mismo costo.

# 6. Diseño del algoritmo

Para el problema de reparto se emplea Dijkstra, porque las conexiones tienen costos distintos. Para el laberinto se emplea BFS, porque todos los movimientos cuestan lo mismo. Ambos algoritmos registran predecesores para reconstruir la ruta final, y mantienen un conjunto o registro de estados visitados para evitar exploraciones repetidas.

## Pseudocódigo — Dijkstra

```
1. Inicializar la distancia del inicio en 0 y las demás en infinito.
2. Insertar el inicio en una cola de prioridad.
3. Extraer el nodo con menor costo provisional.
4. Si es la meta, terminar.
5. Relajar el costo de cada vecino conectado.
6. Si se mejora el costo, guardar predecesor y actualizar la cola.
7. Reconstruir la ruta desde la meta hasta el inicio.
```

## Pseudocódigo — BFS

```
1. Insertar el estado inicial en una cola FIFO.
2. Marcarlo como visitado.
3. Extraer el primer estado.
4. Si es la meta, reconstruir la ruta y terminar.
5. Añadir a la cola cada vecino válido aún no visitado.
6. Guardar su predecesor y continuar hasta hallar la meta o agotar estados.
```

# 7. Código Python comentado

El archivo adjunto **busqueda_2_problemas.py** implementa ambos escenarios sin requerir bibliotecas externas. Para ejecutarlo se necesita Python 3.10 o superior. El programa imprime las rutas, el costo y el número de estados expandidos; además, valida si se encontró solución.

# 8. Resultados de prueba

| Problema | Algoritmo | Solución / salida | Costo | Interpretación |
|---|---|---|---|---|
| Ruta de reparto | Dijkstra | A → C → B → D | 8 km | Menor costo total entre las rutas del grafo. |
| Laberinto | BFS | Ruta de coordenadas calculada al ejecutar el script | Número de pasos | Ruta mínima en movimientos si existe una solución. |

# 9. Análisis de complejidad

**BFS:** con lista de adyacencia, su complejidad temporal es O(V + E) y espacial O(V + E), donde V es el número de estados/nodos y E el número de conexiones. En una cuadrícula, el número de aristas es proporcional al número de celdas, por lo que suele expresarse como O(V) para este tipo de mapa.

**Dijkstra:** con lista de adyacencia y cola de prioridad binaria, la complejidad típica es O((V + E) log V); el almacenamiento del grafo, distancias, predecesores y cola es O(V + E). La implementación de ejemplo usa *heapq* de Python.

**DFS:** aunque no se usa para elegir la solución final de estos dos escenarios, puede explorar un grafo en O(V + E) y espacio O(V) más la representación del grafo. No garantiza la ruta más corta. En general, BFS es preferible para costos uniformes y Dijkstra para costos no negativos diferentes.

## 10. Validación y reproducibilidad

Para reproducir la prueba, guarde el script Python en una carpeta y ejecute `python busqueda_2_problemas.py`. No requiere instalación de paquetes. El resultado depende de los grafos y la cuadrícula definidos dentro del archivo. Los pesos de la red de reparto son datos didácticos, no distancias reales de una ciudad. Para un uso real se necesitarían datos cartográficos, tráfico, restricciones de vehículo y ventanas de entrega.

## 11. Conclusiones

Modelar explícitamente los estados, acciones, transiciones, costos y metas transforma situaciones cotidianas en problemas que pueden resolverse mediante búsqueda. Dijkstra es adecuado para el reparto porque minimiza costos acumulados en aristas ponderadas; BFS es adecuado para el laberinto porque cada movimiento tiene costo unitario. Elegir el algoritmo correcto depende de la estructura del problema y de la métrica que se desea optimizar. Finalmente, un resultado computacional solo es válido respecto al modelo: si los costos, las conexiones o las restricciones no representan la realidad, la ruta óptima del modelo puede no ser la mejor decisión práctica.

## 12. Referencias (formato APA 7)


Russell, S. J., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Bohm, R. A. (2021). *The fundamentals of search algorithms.* Nova Science Publishers.

Python Software Foundation. (s. f.). *heapq — Heap queue algorithm.* Python documentation. https://docs.python.org/3/library/heapq.html


## 13. Declaración de uso de IA

Se utilizó IA generativa como apoyo para estructurar el informe y revisar la explicación del algoritmo. El código es reproducible y debe ejecutarse por el estudiante antes de su entrega; se recomienda verificar los resultados, adaptar el análisis a las indicaciones del docente y mantener esta declaración de acuerdo con las reglas académicas aplicables.